

UNIVERSITATEA „ȘTEFAN CEL MARE" DIN SUCEAVA

Facultatea de Inginerie Electrică și Știința Calculatoarelor

Disciplina: Programare Orientată pe Obiecte



DOCUMENTAȚIE DE PROIECT

Aplicație de gestiune a unui spital

*Sistem orientat pe obiecte pentru pacienți, personal medical,
programări, internări și facturare (C++17)*

Student: Pădurean Gabriel Leonard

Specializarea: Calculatoare, anul II • **Grupa:** 3123A

Temă: 3123A – Aplicație de gestiune a unui spital

Coordonator: Prof. Remus Prodan

Limbaj: C++17 • **Build:** Makefile / CMake • **Versionare:** Git

Suceava, anul universitar 2025–2026

Cuprins

Capitolul 1. Introducere	3
1.1 Contextul și motivația	3
1.2 Obiectivele proiectului	3
Capitolul 2. Cerințele temei	4
Capitolul 3. Tehnologii și instrumente folosite	5
Capitolul 4. Arhitectura aplicației	6
4.1 Organizarea pe module	6
4.2 Structura fizică a proiectului	6
4.3 Fluxul unei operațiuni	6
Capitolul 5. Descrierea claselor	8
5.1 Ierarhia persoanelor	8
5.2 Serviciile medicale (polimorfism)	9
5.3 Programare, Internare, Factură	9
5.4 Managerul Spital	10
5.5 Servicii transversale	10
Capitolul 6. Concepte POO explicate	12
Capitolul 7. Implementarea cerințelor	14
Capitolul 8. Testarea aplicației	15
Capitolul 9. Ghid de compilare și rulare	16
Capitolul 10. Concluzii	17
Capitolul 11. Dezvoltări viitoare	18
Bibliografie și resurse	19

Capitolul 1

Introducere

Acest document descrie, pas cu pas, o aplicație software care administrează activitatea unui spital. Lucrarea este structurată astfel încât să poată fi înțeleasă atât de o persoană cu experiență în programare, cât și de un cititor care întâlnește pentru prima dată concepte de programare orientată pe obiecte.

1.1 Contextul și motivația

Într-un spital există foarte multe informații care trebuie ținute la zi: cine sunt pacienții și ce diagnostic au, ce medici și asistenți lucrează acolo, cine are programare și la ce oră, cine este internat și cât costă, precum și ce facturi se emit. Dacă toate aceste date sunt gestionate manual, apar greșeli: două programări la același medic în același moment, facturi calculate greșit sau pacienți care nu pot fi găsiți rapid.

Aplicația de față rezolvă aceste probleme printr-un program care modelează realitatea spitalului folosind **obiecte**. Fiecare lucru din lumea reală (un pacient, un medic, o programare) devine un obiect în program, cu propriile date și propriul comportament. Această abordare se numește **Programare Orientată pe Obiecte (POO)** și este standardul industrial pentru aplicații de dimensiuni medii și mari.

DEFINIȚIE – POO

Programarea orientată pe obiecte este un stil de programare în care aplicația este construită din „obiecte” care combină **date** (numite atribute) și **operații** (numite metode). Obiectele colaborează între ele pentru a rezolva problema.

1.2 Obiectivele proiectului

Proiectul își propune să demonstreze, într-un mod practic, principalele concepte de POO în limbajul C++:

- modelarea entităților reale prin **clase** și **obiecte**;
- reutilizarea codului prin **moștenire** (o clasă preia caracteristicile alteia);
- tratarea uniformă a unor obiecte diferite prin **polimorfism**;
- protejarea datelor prin **încapsulare**;
- gestionarea situațiilor de eroare prin **excepții**;
- înregistrarea operațiunilor importante prin **logging**;
- verificarea corectitudinii prin **teste unitare**;
- salvarea datelor pe disc prin **persistență în format JSON**.

Pe lângă cerințele obligatorii, au fost implementate și toate cerințele facultative: persistența datelor, șablonul de proiectare *Factory* pentru crearea facturilor și filtrarea pacienților după diagnostic.

Capitolul 2

Cerințele temei

Tema 3123A solicită realizarea unei aplicații de gestiune a unui spital. Cerințele sunt împărțite în obligatorii (necesare pentru funcționalitatea de bază) și facultative (extensii pentru puncte bonus). Tabelul de mai jos arată fiecare cerință și locul unde a fost rezolvată în cod.

Cerință obligatorie	Mod de realizare
Clase: Pacient, Medic, Programare, Factură	Clasele <code>Pacient</code> , <code>Medic</code> , <code>Programare</code> , <code>Factura</code> din directorul <code>src/</code> .
Moștenire: Angajat → Medic, Asistent	Clasa abstractă <code>Angajat</code> este moștenită de <code>Medic</code> și <code>Asistent</code> .
Polimorfism: calcul cost servicii	Metoda virtuală <code>calculeazaCost()</code> din <code>ServiciuMedical</code> , redefinită în clasele derivate.
Excepții personalizate	Ierarhia <code>SpitalException</code> din <code>Exceptii.h</code> (ex: <code>ProgramareInvalidaException</code>).
Logging pentru operațiuni critice	Clasa <code>Logger</code> (Singleton) – internare și emitere factură.
Teste unitare	Fișierul <code>tests/test_spital.cpp</code> – 55 de verificări.
Format cod curat + Git	Cod comentat, organizat pe fișiere; istoric Git cu commit-uri logice.

Tabelul 1: Cerințele obligatorii și modul lor de realizare.

Cerință facultativă	Mod de realizare
Persistență JSON/XML	Modul <code>json</code> scris de la zero + <code>Spital::salveaza()/incarca()</code> .
Pattern Factory pentru facturi	Clasa <code>FacturaFactory</code> cu trei metode de fabricație.
Filtrare pacienți după diagnostic	Metoda <code>Spital::filtreazaDupaDiagnostic()</code> .

Tabelul 2: Cerințele facultative – toate implementate.

Capitolul 3

Tehnologii și instrumente folosite

Proiectul folosește exclusiv tehnologii standard, fără biblioteci externe complexe, pentru a putea fi compilat pe orice sistem cu un compilator modern de C++.

Tehnologie	Rol în proiect
C++17	Limbajul de programare. Versiunea 17 oferă pointeri inteligenți (<code>std::shared_ptr</code>) și expresii lambda folosite în proiect.
STL (Standard Template Library)	Containere și algoritmi din biblioteca standard: <code>std::vector</code> , <code>std::string</code> , <code>std::shared_ptr</code> , <code>std::copy_if</code> .
g++ / MinGW	Compilatorul folosit pentru transformarea codului sursă în program executabil.
Make și CMake	Două sisteme de build care automatizează compilarea (la alegere).
Git + GitHub	Sistem de versionare a codului și găzduire a depozitului.
Format JSON	Format text pentru salvarea și încărcarea datelor spitalului.

Tabelul 1: Tehnologiile și instrumentele utilizate.

DE CE C++ ȘI NU ALTCEVA?

C++ permite controlul fin asupra memoriei și execuției, fiind ideal pentru a învăța conceptele POO „de la firul ierbii”. Spre deosebire de limbaje care ascund detaliile, în C++ vedem clar cum funcționează moștenirea, metodele virtuale și gestiunea obiectelor.

Capitolul 4

Arhitectura aplicației

Aplicația este organizată pe principiul „o responsabilitate pentru fiecare clasă”. Codul este împărțit în patru categorii clare, prezentate în figura de mai jos.

4.1 Organizarea pe module

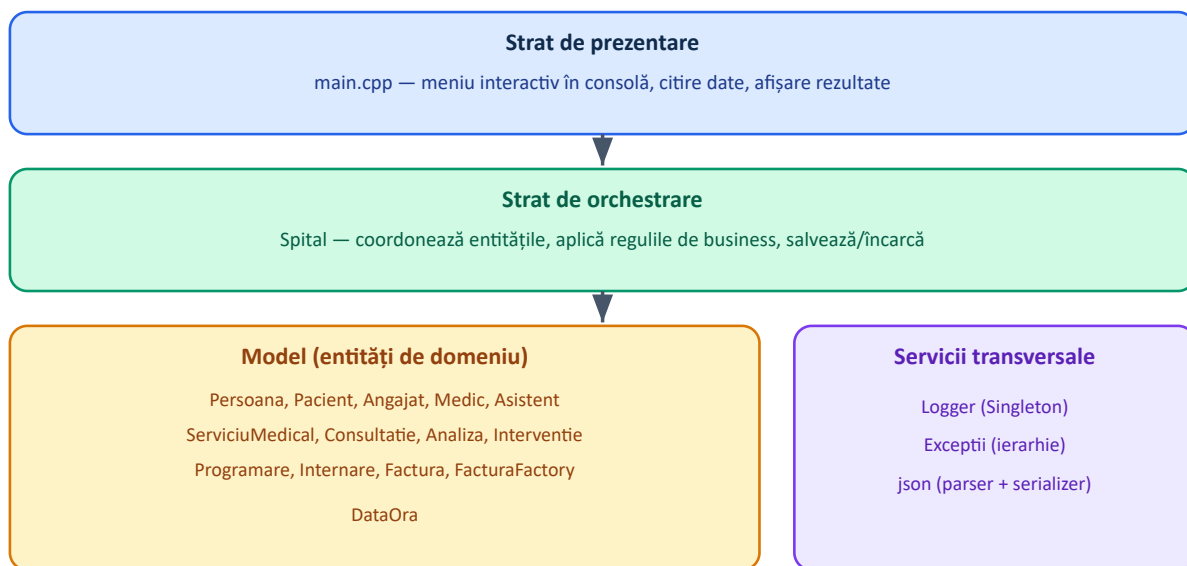


Figura 1: Arhitectura pe straturi a aplicației. Fluxul de control merge de sus în jos: prezentarea cere managerului, managerul folosește modelul și serviciile transversale.

4.2 Structura fizică a proiectului

Fișierele sunt organizate în directoare cu roluri precise, conform bunelor practici:

```

Proiect P00/
├─ src/          # codul sursă (.h cu declarații, .cpp cu implementări)
├─ tests/        # testele unitare
├─ docs/         # documentația (acest fișier, diagrame UML)
├─ data/         # exemplu de date pentru persistență (spital.json)
├─ Makefile      # build cu make
├─ CMakeLists.txt # build cu CMake
└─ README.md     # prezentare și instrucțiuni
  
```

Secvența de cod 1: Structura de directoare a proiectului.

4.3 Fluxul unei operațiuni

Pentru a înțelege cum colaborează componentele, urmărim ce se întâmplă când utilizatorul cere emiterea unei facturi:

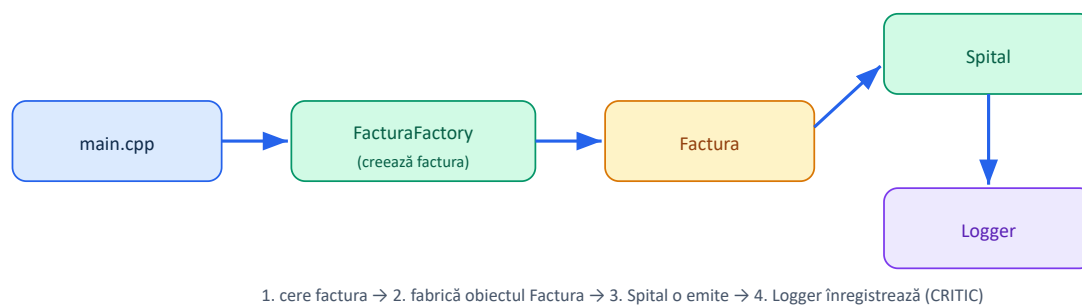


Figura 2: Fluxul emiterii unei facturi: colaborarea dintre prezentare, Factory, model, manager și logger.

Capitolul 5

Descrierea claselor

În acest capitol prezentăm fiecare grup de clase, însoțit de diagrame UML simplificate. În notația UML, o clasă este un dreptunghi împărțit în trei: numele (sus), atributele (mijloc) și metodele (jos). Săgeata cu vârf triunghiular gol ($\triangle$) înseamnă „moștenește din”.

5.1 Ierarhia persoanelor

Toate persoanele din sistem (pacienți și angajați) au în comun un nume și un CNP. De aceea a fost creată o clasă de bază **abstractă** numită *Persoana*. O clasă abstractă este o clasă din care nu se pot crea obiecte direct – ea servește doar ca tipar pentru clasele care o moștenesc.

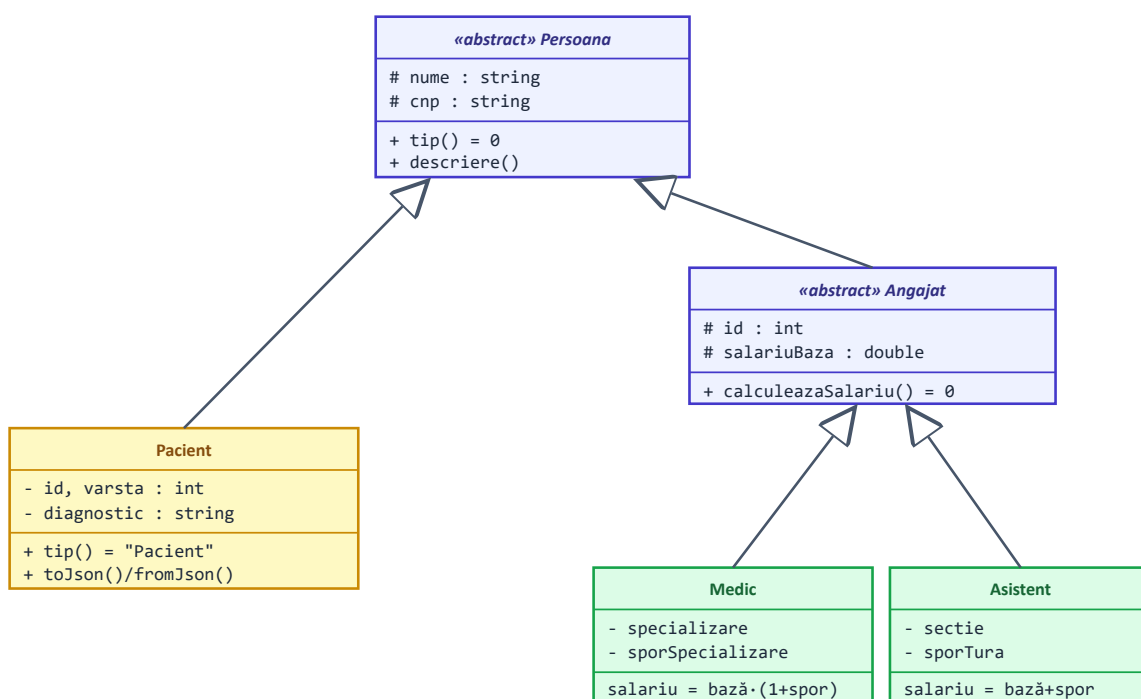


Figura 1: Ierarhia de moștenire a persoanelor. Persoana și Angajat sunt clase abstracte (scrise cu italice).

Clasa *Angajat* introduce o metodă specială: *calculeazaSalariu()*. Aceasta este declarată **virtuală pură** (semnul „= 0”), ceea ce înseamnă că fiecare tip de angajat trebuie să-și calculeze salariul în felul său. Un medic primește un spor procentual de specializare, în timp ce un asistent primește un spor fix de tură.


```
// Medic: salariul crește procentual
double Medic::calculeazaSalariu() const {
    return salariuBaza_ * (1.0 + sporSpecializare_);
}
// Asistent: salariul crește cu un bonus fix
double Asistent::calculeazaSalariu() const {
    return salariuBaza_ + sporTura_;
}
```

Secvența de cod 1: Aceeași metodă, două comportamente diferite – un exemplu de polimorfism.

5.2 Serviciile medicale (polimorfism)

Aici se află „inima” cerinței de polimorfism. Un spital oferă servicii foarte diferite: o consultație simplă, un set de analize de laborator sau o intervenție chirurgicală. Fiecare se taxează după o regulă proprie. Pentru a le trata unitar, toate moștenesc din clasa abstractă `ServiciuMedical`, care declară metoda `calculeazaCost()`.

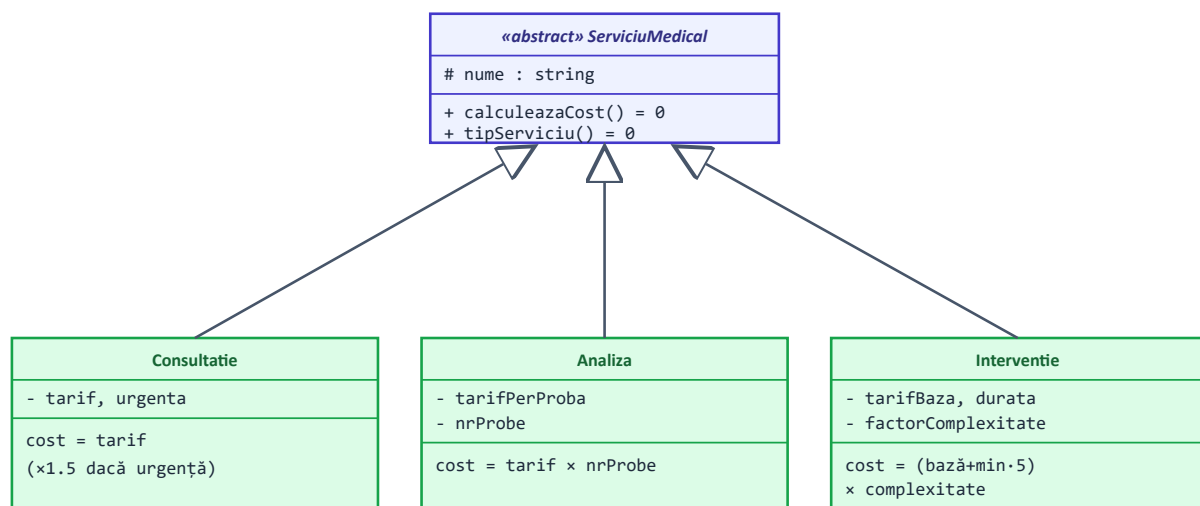


Figura 2: Ierarhia serviciilor medicale. Fiecare clasă derivată redefinește metoda de calcul al costului.

DE CE ESTE UTIL POLIMORFISMUL?

O factură ține o listă de servicii fără să-i pese ce fel sunt. Când calculează totalul, apelează pur și simplu `calculeazaCost()` pentru fiecare; programul alege automat formula potrivită fiecărui obiect. Dacă mâine adăugăm un nou tip de serviciu, factura funcționează fără nicio modificare.

5.3 Programare, Internare, Factură

Aceste clase leagă entitățile între ele:

- **Programare** – asociază un pacient, un medic, o dată/oră și un serviciu. Știe să detecteze dacă intră în conflict cu altă programare (același medic, aceeași oră).

- **Internare** – reprezintă spitalizarea unui pacient într-un salon, pe un număr de zile. Costul = cost pe zi × număr de zile.
- **Factura** – adună o listă de servicii (și, opțional, costul unei internări) și calculează subtotalul, TVA-ul și totalul.

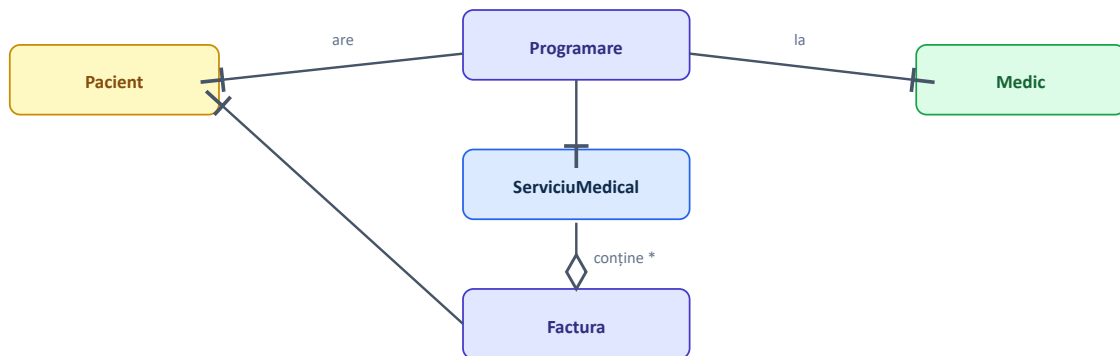


Figura 3: Relațiile dintre entitățile principale (asociere și agregare).

Calculul totalului unei facturi parcurge serviciile și le adună costurile, indiferent de tip:

```

double Factura::subtotal() const {
    double s = costInternare_;
    for (const auto& serviciu : servicii_)
        s += serviciu->calculeazaCost(); // apel polimorfic
    return s;
}
  
```

Secvența de cod 2: Însurarea polimorfică a costurilor pe o factură.

5.4 Managerul Spital

Clasa **Spital** este „creierul” aplicației. Ea deține toate listele (pacienți, angajați, programări, internări, facturi) și aplică regulile de business. Atunci când se adaugă o programare, **Spital** verifică mai întâi dacă medicul nu este deja ocupat la acea oră; dacă da, aruncă o excepție.

```

const Programare& Spital::adaugaProgramare(const Programare& p) {
    for (const auto& existenta : programari_)
        if (existenta.inConflictCu(p))
            throw ProgramareInvalidaException("medic ocupat");
    programari_.push_back(p);
    return programari_.back();
}
  
```

Secvența de cod 3: Detectarea conflictelor de orar înainte de a accepta o programare.

5.5 Servicii transversale

Trei componente ajută toate celelalte clase:

- **Logger** – scrie într-un fișier de jurnal (`spital.log`) momentul și descrierea operațiunilor. Este implementat ca *Singleton* (există o singură instanță în toată aplicația).
- **Exceptii** – o ierarhie de tipuri de erori cu o rădăcină comună (`SpitalException`), pentru a putea trata erorile centralizat.
- **json** – un mic „motor” scris de la zero care transformă obiectele în text JSON și invers, pentru salvarea pe disc.

Capitolul 6

Concepte POO explicate

Acest capitol explică, pe înțelesul tuturor, conceptele de programare orientată pe obiecte folosite și arată exact unde apar în proiect.

6.1 Încapsularea

Ce este: ascunderea datelor interne ale unui obiect și accesul la ele doar prin metode controlate. **De ce contează:** protejează obiectul împotriva valorilor greșite.

În proiect: atributele claselor sunt `private` sau `protected`, iar accesul se face prin getteri. CNP-ul este validat în constructor – nu se poate crea un pacient cu CNP invalid.

6.2 Moștenirea

Ce este: o clasă (derivată) preia atributele și metodele alteia (de bază) și adaugă elemente proprii. **De ce contează:** elimină duplicarea codului. **În proiect:** `Medic` și `Asistent` moștenesc din `Angajat`, iar `Pacient` și `Angajat` moștenesc din `Persoana`.

6.3 Polimorfismul

Ce este: capacitatea de a apela aceeași metodă pe obiecte de tipuri diferite și de a obține comportamente diferite, alese automat la execuție. **În proiect:** `calculeazaCost()` pentru servicii și `calculeazaSalariu()` pentru angajați.

6.4 Abstractizarea

Ce este: definirea unui concept general (o „interfață”) fără detalii de implementare. **În proiect:** clasele abstracte `Persoana`, `Angajat` și `ServiciuMedical` conțin metode virtuale pure pe care derivatele sunt obligate să le implementeze.

6.5 Tratarea excepțiilor

Ce este: un mecanism prin care o eroare „sare” din locul unde a apărut până la codul care o poate trata. **În proiect:** operațiunile invalide aruncă excepții specifice, prinse central în meniul aplicației, astfel încât programul nu se blochează niciodată.

```
try {
    spital.adaugaProgramare(prog);
} catch (const SpitalException& e) {
    std::cout << "[EROARE] " << e.what(); // mesaj prietenos
}
```

Secvența de cod 1: Tratarea centralizată a oricărei erori de domeniu.

6.6 Containere și algoritmi STL

Proiectul folosește `std::vector` pentru liste, `std::shared_ptr` pentru obiecte partajate în siguranță (memoria se eliberează automat) și algoritmul `std::copy_if` pentru filtrare.

6.7 Șabloane de proiectare

Au fost folosite două șabloane consacrate: **Singleton** (Logger – o singură instanță) și **Factory** (FacturaFactory – centralizează crearea facturilor, astfel încât restul codului să nu știe „cum se assemblează” o factură).

Concept POO	Unde apare în proiect
Încapsulare	atribute private/protected + validare în constructori
Moștenire	Persoana → Pacient/Angajat; Angajat → Medic/Asistent
Polimorfism	calculeazaCost(), calculeazaSalariu()
Abstractizare	Persoana, Angajat, ServiciuMedical (metode virtuale pure)
Excepții	ierarhia SpitalException
STL	vector, shared_ptr, copy_if, dynamic_pointer_cast
Șabloane de proiectare	Singleton (Logger), Factory (FacturaFactory)

Tabelul 1: Sinteza conceptelor POO și a locului unde se regăsesc.

Capitolul 7

Implementarea cerințelor

Mai jos arătăm concret cum a fost satisfăcută fiecare cerință, dincolo de simpla bifare.

7.1 Logging pentru operațiuni critice

Internarea unui pacient și emiterea unei facturi sunt considerate operațiuni critice. Ele sunt scrise în jurnal cu nivel `CRITIC` și afișate suplimentar în consolă:

```
[2026-06-03 12:57:32] [CRITIC] EMITERE FACTURA #500 pentru Ion Popescu, total 218 Lei
```

Secvența de cod 1: Exemplu de linie din fișierul de jurnal `spital.log`.

7.2 Persistența JSON (facultativ)

Starea spitalului poate fi salvată într-un fișier `.json` și reîncărcată ulterior. Modulul JSON a fost scris integral în proiect, fără biblioteci externe, și include un analizor (parser) recursiv și un serializator cu indentare.

7.3 Pattern Factory pentru facturi (facultativ)

Clasa `FacturaFactory` oferă trei metode statice de creare: dintr-o programare, dintr-o internare sau dintr-o listă de servicii. Astfel, logica de creare este într-un singur loc.

7.4 Filtrarea pacienților după diagnostic (facultativ)

Metoda `filtreazaDupaDiagnostic()` returnează toți pacienții cu un anumit diagnostic, folosind algoritmul standard `std::copy_if`.

Capitolul 8

Testarea aplicației

Pentru a garanta că aplicația funcționează corect, a fost scris un set de teste unitare. Un test unitar verifică automat că o anumită bucată de cod produce rezultatul așteptat. Testele folosesc un mic cadru propriu (macrourile `VERIFICA` și `VERIFICA_ARUNCA`), fără biblioteci externe.

Grup de teste	Ce verifică
DataOra	respingerea datelor imposibile, anii bisecți, comparațiile
Servicii (polimorfism)	calculul corect al costului pentru fiecare tip de serviciu
Programări	detectarea conflictelor de orar; respingerea celor invalide
Factură	subtotal, TVA și total (servicii + internare)
FacturaFactory	crearea corectă a facturilor pentru fiecare scenariu
Filtrare	returnarea pacienților cu un diagnostic dat
Validare CNP	aruncarea excepției pentru CNP invalid
JSON	parsare, serializare și „dus-întors” (round-trip)

Tabelul 1: Acoperirea testelor unitare.

REZULTAT

La rularea testelor: **55 de verificări, 0 eșecuri** – „TOATE TESTELE AU TRECUT”. Programul returnează codul de ieșire 0 la succes și 1 la eșec, fiind potrivit pentru integrare continuă (CI).

Capitolul 9

Ghid de compilare și rulare

Proiectul oferă trei variante de compilare, la alegere.

9.1 Varianta cu Make (Linux / WSL / MinGW)

```
make          # compilează aplicația  
make run      # compilează și rulează  
make test     # rulează testele unitare  
make clean    # șterge fișierele generate
```

Secvența de cod 1: Comenzile sistemului de build Make.

9.2 Varianta cu CMake (portabil)

```
cmake -S . -B build  
cmake --build build  
ctest --test-dir build # rulează testele
```

Secvența de cod 2: Comenzile pentru CMake.

9.3 Compilare directă cu g++ (Windows)

```
g++ -std=c++17 -Wall -Wextra -Isrc src/*.cpp -o build/spital.exe  
build\spital.exe
```

Secvența de cod 3: Compilare directă, fără sistem de build.

9.4 Exemplu de rulare

La pornire, aplicația încarcă date demonstrative și afișează un meniu cu 11 opțiuni (afișare pacienți, adăugare programare, internare, emitere factură, filtrare, salvare/încărcare JSON etc.). Toate erorile sunt tratate elegant, fără ca programul să se oprească brusc.

Capitolul 10

Concluzii

Proiectul realizează integral o aplicație de gestiune a unui spital, respectând toate cerințele obligatorii și implementând, în plus, toate cerințele facultative. Pe parcursul dezvoltării au fost puse în practică, într-un mod coerent, principiile fundamentale ale programării orientate pe obiecte.

Punctele forte ale soluției sunt:

- **arhitectură curată**, cu separare clară între model, orchestrare, prezentare și servicii transversale;
- **extensibilitate** – datorită polimorfismului, adăugarea unui nou tip de serviciu sau de angajat nu afectează codul existent;
- **robustețe** – validări sistematice și o ierarhie de excepții care împiedică stările invalide;
- **verificabilitate** – 55 de teste unitare automate;
- **independență** – nu folosește biblioteci externe, deci compilează oriunde.

Lucrul la acest proiect a consolidat înțelegerea modului în care conceptele teoretice de POO (moștenire, polimorfism, abstractizare) se traduc în cod real, întreținut și testat.

Capitolul 11

Dezvoltări viitoare

Aplicația poate fi extinsă în mai multe direcții:

- **Persistență completă** – salvarea și reîncărcarea programărilor și internărilor, cu refacerea legăturilor dintre entități pe baza identificatorilor.
- **Indexare rapidă** – folosirea unui `std::unordered_map` pentru căutarea pacienților în timp constant.
- **Rapoarte statistice** – venituri pe secție, grad de ocupare a saloanelor, număr de consultații pe medic.
- **Interfață grafică sau API web** – construite peste aceeași logică de domeniu, fără a rescrie nimic.
- **Validare avansată a CNP-ului** – verificarea cifrei de control.

Capitolul 12

Bibliografie și resurse

1. Bjarne Stroustrup, *The C++ Programming Language*, ediția a 4-a, Addison-Wesley, 2013.
2. Bjarne Stroustrup, *Programming: Principles and Practice Using C++*, ediția a 2-a, Addison-Wesley, 2014.
3. Documentația oficială C++: <https://en.cppreference.com>
4. Erich Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 (șabloanele Singleton și Factory).
5. Standardul ISO/IEC 14882:2017 – limbajul C++17.
6. Documentația CMake: <https://cmake.org/documentation>
7. Specificația formatului JSON: <https://www.json.org>
8. Depozitul proiectului pe GitHub: <https://github.com/leopg1/Proiect-POO>